

Alternative path (when you want “portable installs”)

YoWASP provides WebAssembly-based packages for Yosys/nextpnr tool variants (useful in some setups). It’s not required for this Fast Track, but it’s good to know it exists.

Constraints 101: pin mapping is where projects go to die

Your HDL has signal names like `led`, `btn`, `uart_tx`. Your FPGA has physical pins like `A3`, `T6`, `P4`, etc.

A constraints file is the contract between your RTL and the real board.

iCE40 constraints: PCF

nextpnr’s iCE40 docs describe PCF constraints and how they’re passed via `--pcf`. The basic syntax uses `set_io` for mapping signals to pins.

Example pattern:

```
set_io led 3
set_io btn 4
```

(Your pin numbers will differ by board.)

ECP5 constraints: LPF

nextpnr-ecp5 supports LPF pin constraint files (passed with `--lpf`), and the manpage documents this option.

Common LPF pattern:

```
LOCATE COMP “led_o” SITE “T6”;
IOBUF PORT “led_o” IO_TYPE=LVCNMOS33;
```

(Again, exact site names depend on board.)

Step-by-step: how to create constraints without guessing

1. Find your board’s official pinout/constraints reference (often shipped as a `.pcf/.lpf` or in docs).
2. Identify one LED pin and one button pin.
3. Map your RTL top-level port names to those pins.
4. Start with one LED only. Don’t map 12 signals in your first run.
5. If build fails, you have fewer variables to debug.

If you want a very “show me an example” reference, there are community write-ups that show full PCF and LPF examples end-to-end (useful as examples, not as authoritative pinouts for your specific board).

“Hello FPGA” build: Blinky, but proper

We’ll do the universal first FPGA project: blink an LED using a counter. This is ideal because:

- it proves synthesis + P&R + bitstream generation
- it proves your constraints are correct
- it gives you a stable, visible result

The design goal

- Use the board’s clock
- Toggle LED at a human-visible rate (say ~1 Hz)

Step 1: create the project folder

Create:

- `src/` for RTL
- `constraints/` for PCF or LPF
- `build/` for outputs

Step 2: write blinky.v (RTL)

Use a parameterized counter so you can adapt to different clocks.

```
module blinky #(
    parameter integer CTR_WIDTH = 24
)()
    input wire clk,
    input wire rst_n, // active-low reset
    // (tie high if you don't have a reset button)
    output reg led

    reg [CTR_WIDTH-1:0] ctr;

    always @(posedge clk) begin
        if (!rst_n) begin
            ctr <= 0;
            led <= 1'b0;
        end else begin
            ctr <= ctr + 1'b1;
            led <= ctr[CTR_WIDTH-1]; //
            // MSB toggles slowest
        end
    end
endmodule
```

Why this works: the MSB of a counter toggles slowly. Adjust `CTR_WIDTH` to tune blink speed.

